How Claude Code Works

A beginner's guide — from your prompt to the model's next token

Worked examples drawn from the CeReS codebase

Contents

Part 1 — The Agent

How Claude Code processes a request like *"refactor the authentication code to use async/await."*

The big picture

Claude Code is an **agent**: it doesn't just answer, it takes actions (reading files, editing, running commands) in a loop until your task is done. Think of it as a developer who reads your codebase, makes a plan, edits files, and checks their work.

Step 1 — Claude reads your prompt + its context

When you type a prompt, Claude receives more than just your words:

- Your message
- A **system prompt** (rules about how to behave, its tools, your OS/git info)
- Any files you have open in the IDE
- `CLAUDE.md` (project instructions), recent git history, etc.

So "refactor the code" is interpreted *in the context of your actual project*, not in a vacuum.

Step 2 — Claude decides it needs more information

It almost never edits blindly. For "refactor the auth code" it first asks: *Where is the auth code? What does it look like? What depends on it?* Then it uses **tools** to find out:

- **Search tools** (`Grep`, `Glob`) → "find files matching `*auth*`", "grep for `login(`"
- **Read tool** → open the relevant files and read them
- Sometimes it launches a sub-agent to explore in parallel

Step 3 — The "agent loop" (the core mechanic)

Claude works in a **loop**:

```
1. Think        → "I need to see auth.py"
2. Call a tool  → Read auth.py
3. Get result   → (file contents come back)
4. Think again  → "Now I understand it; I'll change these 3 functions"
5. Call a tool  → Edit auth.py
6. Get result   → (edit confirmed)
7. Repeat...    → until the task is complete
```

Each tool result feeds the next decision. It's not one giant answer — it's many small, verifiable steps.

Step 4 — Planning (for anything non-trivial)

For a real refactor, Claude often **makes a plan first** — sometimes showing it to you for approval before touching code. This prevents wasted work. For big tasks it tracks the steps as a to-do list.

Step 5 — Making the changes

- **Edit** → find-and-replace an exact chunk of code (it must have read the file first)
- **Write** → create or overwrite a whole file

It matches your existing style (naming, formatting, patterns) rather than imposing its own.

Step 6 — Verifying its own work

A good refactor isn't done when the edit is saved. Claude will typically run the **tests** and the **linter/type-checker**, read the errors, fix them, and re-run — looping until green.

Step 7 — Reporting back

Finally it tells you what it did, honestly — what changed, what passed, what's still broken, and any caveats.

A concrete mini-example

Claude does	Why
`Grep "getUser"`	find where it lives + who calls it
`Read userService.ts`	understand current code
`Edit` — wrap in try/catch	make the change
`Bash: npm test`	verify nothing broke
Reads a failing test	catches a mistake
`Edit` — fix it	correct it
"Done — added error handling, all tests pass"	report

Key mental models

1. **It's a loop, not a single reply.** Act → observe → act again.
2. **Tools are how it touches the world.** It must call the Edit tool; you see every call.
3. **You're in control.** It asks before risky actions; you can interrupt anytime.
4. **Context is everything.** A specific prompt beats a vague "refactor the code."
5. **It verifies.** Good runs end with tests/lint passing.

Writing better prompts

- **Name the file/function**: "refactor `handler.py`" beats "refactor the code"
- **State the goal**: "…to reduce duplication" or "…to use async/await"
- **Set constraints**: "don't change the public API", "keep it backwards-compatible"
- **Ask for a plan first** if it's big
- **One clear task per message** for complex work

Part 2 — The Model's Mind

How the LLM beneath Claude actually "reads" a `.py` file and "knows" how to refactor.

First: the model doesn't "read" like a human

There's no understanding of meaning the way you have. What's really happening is **extremely sophisticated pattern prediction**. The model's one skill is: *given some text, predict the next chunk of text.* Understanding, refactoring, and reasoning all emerge from doing that incredibly well.

Step 1 — Your text becomes numbers (tokenization)

The model can't process letters. Your prompt and the `.py` file are chopped into **tokens** — small pieces (roughly ¾ of a word, or a symbol).

```
"def login(user):"  →  ["def", " login", "(", "user", "):"]
```

Each token maps to an ID:

```
["def", " login", "(", "user", "):"]  →  [1050, 6923, 7, 1428, 4390]
```

A Python file is just more tokens. To the model, code and English are the **same kind of thing** — that's why one model handles both.

Step 2 — Tokens become vectors (embeddings)

Each token ID becomes an **embedding** — a long list of numbers, i.e. coordinates in a huge "meaning space." Tokens with similar meaning land near each other: `login` and `authenticate` are close; `login` and `banana` are far apart.

Step 3 — Attention: relate every token to every other

This is the heart of the Transformer. **Attention** lets each token "look at" all the others and decide which matter. Reading:

```
def login(user):
    token = get_token(user)
    return token
```

with the prompt "make login handle errors", attention wires up: "errors" ↔ `get_token` / `return`; the `token` in the body ↔ the `token` in the return. It does this across **dozens of layers**, each building

richer understanding. Nobody programmed those rules — they emerged.

Step 4 — Where the "knowledge" comes from (training)

Before you used it, the model read millions of files/docs and played one game billions of times: *hide the next token, guess it, check, adjust.* To win, it had to internalize: valid syntax, that `try:` is followed by `except:`, that error handling wraps risky calls, that "refactor to async" means adding `async` / `await` and updating callers. "Knowing how to refactor" = those patterns compressed into billions of weights.

Step 5 — Generating the refactor (one token at a time)

```
It has: [your prompt] + [file contents] + [instruction to edit]
Predict next token → "def"
Predict next       → " login"
Predict next       → attention says errors wanted → "try"
... each token informed by everything before it
```

Step 6 — In-context learning (why it adapts to YOUR code)

The model wasn't trained on your codebase, yet it matches your style. This is **in-context learning**: the tokens you just gave it (your file) steer the prediction. If your file uses `snake_case` and returns dicts, attention pulls generation toward `snake_case` and dicts — no retraining needed.

The honest caveats

- **It's prediction, not comprehension.** No beliefs or intentions — pattern-completion that *looks* like understanding.
- **It can be confidently wrong.** That's *why* the agent loop runs tests.
- **No memory between messages** except the text in the current context window.

> *The model turns your words and code into numbers, uses "attention" to figure out how everything relates, and — drawing on patterns learned from billions of code examples — predicts the refactored code one token at a time, imitating your specific file as it goes.*

Part 3 — Live Trace

One real line from the CeReS repo, traced through the model — all the way to the fix we committed.

The line, from `backend/lambdas/certificate_api/amq_consumer.py`:

```
event = create_mq_event([body])
```

Stage 1 — Tokenization

token	what it is
`\n        `	newline + indentation
`event`	a word
`=`	space + equals
`create`	space + word
`_mq`	subword piece
`_event`	subword piece
`([`	two symbols merged
`body`	a word
`])`	two symbols merged

`create_mq_event` is **three tokens**, not one — the model reassembles meaning from pieces, which is why it handles names it's never seen. IDs (illustrative):

```
[198, 6531, 796, 2251, 62, 76, 80, 62, 15596, 26933, 2618, 12962]
```

Stage 2 — Embeddings (closeness learned in training)

```
similarity(event, message)  = 0.7   ← related
similarity(create, build)   = 0.8
similarity(body, payload)   = 0.6
similarity(event, banana)   = 0.02  ← unrelated
```

Stage 3 — Positional info

`create_mq_event([body])` ≠ `body([create_mq_event])`. Each vector gets "where am I" added, so order is preserved.

Stage 4 — Attention (the actual "thinking")

For the token `body`, attention scores across the line:

```
body attends to:
   create   ████████▒▒  0.42   ← "I'm the argument to this call"
   event    ███▒▒▒▒▒▒▒  0.18   ← "I end up assigned to event"
   [        ██▒▒▒▒▒▒▒▒  0.12
   self     █▒▒▒▒▒▒▒▒▒  0.05
   =        ▒▒▒▒▒▒▒▒▒▒  0.02
```

Repeated across many layers, understanding deepens:

```
Layer 1  : "event is a variable being assigned"
Layer 5  : "create_mq_event is a function call returning something"
Layer 12 : "this builds an ActiveMQ event from a message body"
Layer 30 : "matches how message_handler expects its input"
```

Stage 5 — Connecting to the real refactor

The next line was `message_handler(event, None)` and pyright complained: *None not assignable to LambdaContext.* The model's context held, all attended together:

```
[ error text: "None cannot be assigned to LambdaContext" ]
[ code:        message_handler(event, None) ]
[ signature:   def message_handler(event, _: LambdaContext) ]
```

Attention links the error's `LambdaContext` to the code's `None`; weight-memory supplies the trained pattern *"type error: X not assignable to Y → wrap with cast(Y, X)."*

Stage 6 — Generation (one token at a time)

Predicting the token after `message_handler(event,`:

```
next-token probabilities:
   cast       0.61   ██████████▒▒▒▒▒▒▒▒▒▒
   None       0.24   ████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒
```

```
   context    0.08   ██▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒
   ...        (thousands more, tiny)
```

It picks `cast`, feeds back, predicts again:

```
after "cast"     → "("        0.95
after "cast("    → "Lambda"   0.79   ← type pulled from the error msg
after "Lambda"   → "Context"  0.98
after "Context"  → ","        0.90
after ","        → " None"    0.88
after " None"    → ")"        0.97
```

Assembled: `cast(LambdaContext, None)` — exactly the fix committed.

Stage 7 — In-context learning matched YOUR file

There was no `from typing import cast` in the visible tokens, so attention linked the newly-generated `cast` usage to "no cast import present → emit one." That's why the edit also added the import — pure prediction, steered by what was and wasn't in your file.

The whole journey in one picture

```
"event = create_mq_event([body])"
        |
   [tokenize]      → [198, 6531, 796, 2251, 62, ...]
        |
   [embed]         → each token → 4096-number vector in meaning-space
        |
   [+ position]    → add "where am I"
        |
   [attention ×N]  → every token relates to every other, across layers
        |              (body → argument of create_mq_event → becomes event)
        |
   [weights recall]→ training pattern: "cast fixes this type error"
        |
   [generate]      → probability over 100k tokens, pick best, repeat
        |
   "cast(LambdaContext, None)"  + "from typing import cast"
```

> *Everything — reading the file, understanding the type error, knowing `cast` is the fix, matching your import style — is the same operation repeated: turn tokens into vectors, use attention to relate them, and predict the next token using patterns compressed from billions of examples.*

> *There is no separate "refactoring module." Comprehension and code-writing are both next-token prediction on top of geometry (embeddings) and relationships (attention).*